



[AI 반도체 스터디 #1]

시스톨릭 어레이 (Systolic Array)

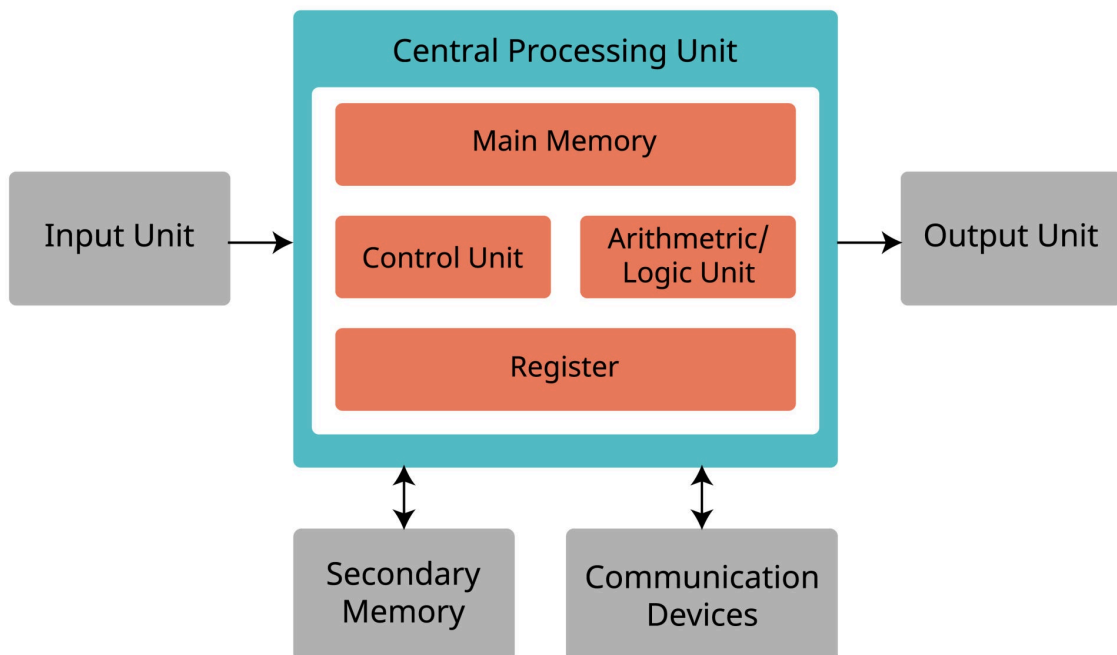
요즘 소프트웨어 알고리즘과 하드웨어 설계 사이에서 “내가 뭘 더 좋아할까” 많은 고민을 해왔음. 이번 프로젝트를 통해 소프트웨어 관련 기술들과 하드웨어 관련 기술들을 매주 조사하면서 생각을 굳힐 계획임.

일단 이번 조사를 통해 “아무리 똑똑한 소프트웨어가 있어도, 이를 뒷받침할 하드웨어 구조의 혁신이 없으면 무용지물이구나” 라는 생각을 함.

왜 이런 생각을 했는지 설명하기 위해서 최신 AI 반도체(NPU)의 심장이라 불리는 '**시스톨릭 어레이(Systolic Array)**' 아키텍처를 나만의 언어로 풀어내어 공유하려고 함.

시스톨릭 어레이 자체는 개념적으로는 옛날 기술이지만, AI 시대를 맞아 가장 화려하게 부활한 가장 트렌디한 기술이라고 봐서 조사해봄.

1. 기술이 등장한 배경과 문제의식 (Why?)



기존 구조의 한계: "데이터를 나르다가 지친다"

Chat GPT나 딥러닝 알고리즘의 본질은 결국 수십억 번의 **행렬 곱(Matrix Multiplication)** 연산이잖아 희찬아 정길아 그치?

하지만 기존의 컴퓨터 구조(폰 노이만 구조)는 연산을 하나 할 때마다 메모리에서 데이터를 가져오고 ➡ 연산기(ALU)에서 계산하고 ➡ 다시 메모리에 저장하는 방식을 씬.

그래서 알고리즘 코드를 아무리 기막히게 짜도, 결국 연산 속도가 메모리에서 데이터를 읽어오는 속도를 따라가지 못하는 **메모리 벽(Memory Wall) 병목현상**에 부딪히게 됨. 배보다 배꼽이 더 큰 상황임.

💡 문제의식 : "메모리에 안 가고 자기들끼리 계산할 수 없을까?"

1978년, 카네기 멜런 대학교의 H. T. 쿡 교수(솔직히 누군지 모름)는 이 비효율을 해결하기 위해 "데이터를 메모리에서 매번 읽어오지 말고, 연산기들을 그물망처럼 촘촘히 연결해서 데이터가 그 사이를 통과하며 연속으로 계산하게 만들자!" 라는 아이디어와 함께 독특한 하드웨어 구조를 제안함.

2. 최신 기술의 개념과 구조 이해 (What & How)

'시스톨릭(Systolic)'은 '심장 수축'을 뜻하는 의학 용어임. 심장이 신선한 피를 펌프질해서 온몸의 혈관으로 순환시키듯이, 데이터가 반도체 내부의 연산기 배열을 멈추지 않고 물 흐르듯 통과하는 구조라고 보면 됨.

- **2차원 연산기 그물망 (PE Array)** : 칩 내부에는 무수히 많은 작은 연산기(PE: Processing Element)가 바둑판처럼 배열되어 있음.
 - **맥박 메커니즘 (Dataflow)**:
 1. 행렬 A의 데이터는 왼쪽에서 오른쪽으로 흐름.
 2. 행렬 B의 데이터는 위에서 아래로 흐름.
 3. 중간에 만난 연산기(PE)들이 곱하고 더한 뒤, 결과를 굳이 메모리로 보내지 않고 **바로 옆과 아래에 있는 연산기에게 토스함**.
 - **핵심 이점**: 데이터를 메모리에서 **단 한 번만** 읽어오면, 수백 개의 연산기가 달라붙어 징검다리 건너듯 최종 행렬 값을 완성함. 메모리 접근 횟수가 극적으로 줄어들어 전력과 속도를 모두 잡을 수 있음.
-

3. 실제 활용 사례와 한계 분석 (Case & Limit)

🚀 성공 사례 : 40년 만의 화려한 부활, 구글 TPU & 엔비디아 텐서코어

이 기술은 발표 당시에는 거대한 행렬을 풀 소프트웨어 수요가 없어서 이론 속에 묻혀 있었음. 하지만 2016년, 구글이 딥러닝 전용 반도체인 **TPU(Tensor Processing Unit) v1**에 이 구조를 전면 도입하면서 세상에 점차 들어남. 기존 범용 하드웨어 대비 **연산 속도는 30배, 전력 효율은 80배 이상** 끌어올리고 알파고의 심장 역할을 함. 현재 엔비디아 최신 AI 칩(H100, B200 등)의 핵심인 '텐서 코어'도 이 구조를 기반으로 함.

⚠️ 치명적인 한계 (Limit)

- **낮은 유연성 (Flexibility 부족):** 이 하드웨어는 '정방형 행렬 곱'을 할 때만 100% 성능을 냄. 만약 AI 모델의 형태가 직사각형이거나, 특이한 레이어(ex : 스parse 매트릭스)가 들어오면 **연산기의 절반 이상이 놀게 되는(Zero-utilization) 비효율**이 발생해버림. 즉, 특정 AI 알고리즘에만 극도로 최적화된 하드웨어라는 양날의 검 느낌임.

4. 개인적인 해석과 생각의 변화 기록 (Insight)

이번 조사를 하기 전까지 나는 "더 좋은 AI를 만들려면 소프트웨어 모델의 알고리즘 복잡도를 줄이거나 수학적 트릭을 써야 한다"고만 생각했고, 하드웨어는 그저 코드를 실행해주는 '도구' 라고 봤음.

하지만 시스톨릭 어레이를 조사하면서 생각이 좀 바뀜. 소프트웨어가 직면한 물리적 한계(메모리 대역폭, 전력)를 **하드웨어의 공간적 배치와 데이터 흐름(Dataflow) 설계**라는 '**구조적 혁신**'으로 넘을 수 있다는 것을 알게됨. ○○ 당연한 거긴 해

이제는 알고리즘(SW)과 아키텍처(HW)를 분리해서 보는 것이 아니라, "내가 짤 반도체 로직(Verilog)이 이 소프트웨어의 행렬을 얼마나 낭비 없이 꽉 채워 연산할 수 있을까?"라는 HW-SW 협력 설계(Co-design)의 관점으로 보려고 노력 중인데.. 어려움. 처음이니까 봐주셈.

💡 스터디원들에게 하는 질문 및 논의(Discussion)

1. **하드웨어 vs 소프트웨어 :** 시스톨릭 어레이처럼 "특정 AI 연산에만 극대화된 하드웨어 칩을 만드는 것"과 "기존 하드웨어에 맞춰 알고리즘을 극한으로 깎는 소프트웨어 최적화" 중, 향후 5년 내 가성비와 상용화 측면에서 시장을 지배할 핵심 치트키는 어느 쪽일까?

2. **한계 극복 아이디어** : 시스톨릭 어레이는 고정된 그물망 구조 때문에 AI 모델이 조금만 바뀌어도 효율이 떨어진다는 한계가 있음. 하드웨어 엔지니어 관점에서 이 '노는 연산기'들을 없애기 위해 어떻게 해야될까?

정길아 (희찬아) 코멘트 남겨줘 (희찬이는 남기던지 말던지...)